

Explicación Detallada del Script de Predicción del Índice S&P 500 (Semana 13)

Modelos ETS, ARMA sobre log-retornos, tratamiento de atípicos,
validación *walk-forward* y ensambles ponderados

Documento técnico generado para acompañar el script en R

21 de julio de 2026

Índice

1. Introducción y visión general del proyecto	3
2. Librerías utilizadas	4
3. Carga de la serie de tiempo	5
3.1. Descarga de los datos	5
3.2. Construcción del <code>tsibble</code>	5
4. Tratamiento de atípicos	7
4.1. Estrategia 1: <code>tsclean()</code> (método de Hyndman)	7
4.2. Estrategia 2: eliminación manual de ventanas de fechas	8
5. Parámetros globales y funciones de evaluación	9
5.1. Parámetros globales	9
5.2. Función de pérdida de Winkler	9
5.3. Indicador de cobertura	10
5.4. Combinación de las funciones de pérdida	11
6. Función pronosticar_individual	13
6.1. Selección del conjunto de entrenamiento	13
6.2. Modelo ETS (suavizamiento exponencial)	13
6.3. Modelo Naive (paseo aleatorio)	14
6.4. Modelo ARMA automático sobre log-retornos	14
7. Función evaluar_modelos: validación <i>walk-forward</i>	17
8. Función evaluar_ensambles	21
9. Función tabla_final_MSE	25
10. Escenarios: evaluación de modelos individuales desde 2023	26

11. Validacion por combinaciones de modelos (ensambles)	28
12. Selecccion de modelos diferenciada por horizonte	30
13. Generacion de pronosticos finales e intervalos de confianza	33
14. Conclusion metodologica	34

1 Introducción y visión general del proyecto

Este documento explica, línea por línea y bloque por bloque, el script de R “**Script semana 13**”, cuyo objetivo es construir y validar modelos de pronóstico de corto plazo (horizonte $h = 5$ días hábiles) para el índice bursátil **S&P 500** (ticker `^GSPC`), usando información diaria desde 2015 (y una ventana de trabajo específica desde 2023).

El código **no se modifica** en ningún momento; este documento se limita a reproducir cada fragmento relevante y a explicar:

- **Qué hace** cada línea o bloque de código.
- **Por qué** se hace de esa manera (justificación estadística y de buenas prácticas en pronóstico de series de tiempo).
- El **sustento teórico** necesario para entender la técnica utilizada (suavizamiento exponencial, modelos ARMA, validación cruzada para series de tiempo, funciones de pérdida para intervalos de predicción, y combinación de pronósticos).

A un nivel macro, el script sigue esta hoja de ruta:

1. **Carga de datos:** se descargan precios ajustados de cierre del S&P 500 desde Yahoo Finance y se estructuran como una serie de tiempo (`tsibble`).
2. **Tratamiento de atípicos:** se generan tres versiones de la serie – la original, una versión “limpiada” automáticamente con `tsclean()`, y una versión donde se eliminan manualmente intervalos de fechas conflictivos.
3. **Funciones de evaluación:** se definen funciones para calcular el error cuadrático, la pérdida de Winkler (para intervalos de confianza) y el conteo de observaciones fuera del intervalo.
4. **Modelos individuales:** ETS (suavizamiento exponencial), Naive (paseo aleatorio) y ARMA automático sobre *log-retornos*.
5. **Validación *walk-forward*** (origen móvil): cada modelo se evalúa repetidamente hacia adelante en el tiempo, sobre las tres versiones de la serie y con ventana expansible.
6. **Ensamblados:** se combinan modelos con diferentes tratamientos de atípicos usando ponderaciones (iguales o inversamente proporcionales al MSE), incluyendo una estrategia diferenciada por horizonte de pronóstico ($h = 1, 2, 3$ vs. $h = 4, 5$).
7. **Pronóstico final:** se genera el pronóstico a 5 días con el esquema de combinación ganador, junto con intervalos de confianza contruidos bajo el supuesto de normalidad de los errores de validación.

2 Librerías utilizadas

```
1 library(quantmod)
2 library(ggplot2)
3 library(fpp3)
4 library(tsibble)
5 library(dplyr)
6 library(forecast)
7 library(tidyverse)
8 library(gridExtra)
9 library(nnlsl)
```

- **quantmod**: provee la función `getSymbols()` para descargar series financieras (precios, volúmenes) desde fuentes como Yahoo Finance, y funciones auxiliares como `Ad()` para extraer el precio de cierre ajustado.
- **ggplot2**: sistema de gráficos basado en la “gramática de gráficos” (Wilkinson), utilizado implícitamente por otras librerías del ecosistema **tidyverse**/**fpp3** para graficar series y pronósticos.
- **fpp3**: paquete que acompaña el libro *Forecasting: Principles and Practice* (3.^a ed., Hyndman & Athanasopoulos). Reúne **fable** (modelos de pronóstico en formato **tidy**), **feasts** (análisis exploratorio de series) y **tsibble**.
- **tsibble**: estructura de datos “tidy” para series de tiempo, análoga a un **data.frame** pero con una columna índice temporal explícita (**index**), lo que permite operaciones coherentes con fechas o con un índice ordinal como en este script (**t**).
- **dplyr**: manipulación de datos (**filter**, **mutate**, **select**), parte del **tidyverse**.
- **forecast**: contiene `auto.arima()` y `tsclean()`, funciones clásicas (no “tidy”) de Hyndman para ajuste automático de modelos ARIMA y limpieza de atípicos.
- **tidyverse**: colección de paquetes para ciencia de datos (incluye **dplyr**, **purrr**, **tibble**, etc.).
- **gridExtra**: permite organizar múltiples gráficos de **ggplot2** en una sola rejilla (`grid.arrange()`).
- **nnlsl**: resuelve problemas de mínimos cuadrados no negativos (*Non-Negative Least Squares*). Aunque no se usa de forma explícita en el cuerpo mostrado del script, suele emplearse para estimar pesos óptimos y no negativos en la combinación de pronósticos (ensambles), lo cual es coherente con la sección de ensambles del script.

3 Carga de la serie de tiempo

3.1 Descarga de los datos

```
1 simbolo      <-  "~GSPC"
2 fecha_inicio <-  "2015-01-01"
3 fecha_fin    <-  "2026-06-19"
4
5 datos_sp500 <- getSymbols(
6   Symbols    = simbolo,
7   src        = "yahoo",
8   from       = fecha_inicio,
9   to         = fecha_fin,
10  auto.assign = FALSE
11 )
12
13 adj_close <- Ad(datos_sp500)
```

Explicacion linea por linea:

- `simbolo <- "~GSPC"`: `^GSPC` es el ticker que Yahoo Finance utiliza para el indice S&P 500. El simbolo `^` es la convencion de Yahoo para indices bursatiles (en oposicion a acciones individuales).
- `fecha_inicio` y `fecha_fin`: delimitan la ventana temporal de descarga: desde el 1 de enero de 2015 hasta el 19 de junio de 2026.
- `getSymbols(...)`: funcion de `quantmod` que se conecta a la fuente `src = "yahoo"` y descarga un objeto de series de tiempo tipo `xts` con columnas Open, High, Low, Close, Volume y Adjusted.
- `auto.assign = FALSE`: por defecto, `getSymbols()` asignaria automaticamente un objeto llamado `GSPC` al entorno global. Al fijar este argumento en `FALSE`, el resultado se captura explicitamente en `datos_sp500`, lo cual es una buena practica para evitar efectos colaterales y tener control total sobre el objeto.
- `Ad(datos_sp500)`: extrae unicamente la columna de **precio de cierre ajustado** (*Adjusted Close*). Este precio ajustado corrige el precio de cierre nominal por eventos corporativos (dividendos, *splits*), por lo que es la serie recomendada para analisis de retornos y modelado financiero, ya que refleja el verdadero rendimiento economico del activo.

3.2 Construccion del tsibble

```
1 adj_close_dataframe <- data.frame(
2   Fecha      = zoo::index(adj_close),
3   AdjClose   = as.numeric(adj_close)
4 )
5
6 adj_close_2015 <- adj_close_dataframe |>
7   mutate(t = row_number()) |>
8   as_tsibble(index = t)
9
10 # Ventana desde 2023 (unica que se usa esta semana)
```

```
11 adj_close_2023 <- adj_close_2015[adj_close_2015$Fecha >= "2023-01-01", ]  
    |>  
12 mutate(t = row_number()) |>  
13 as_tsibble(index = t)
```

- `zoo::index(adj_close)`: los objetos `xts` (usados por `quantmod`) estan contruidos sobre `zoo`; `index()` extrae el vector de fechas asociado a cada observacion.
- Se construye un `data.frame` plano con dos columnas: `Fecha` (fecha calendario) y `AdjClose` (precio ajustado numerico), desacoplando los datos del formato `xts`.
- `mutate(t = row_number())`: se crea un indice **ordinal** `t = 1, 2, 3, ...`, que numera consecutivamente los dias habiles disponibles. Esto es clave: como el mercado bursatil **no opera todos los dias del calendario** (no hay fines de semana ni dias festivos), usar la fecha calendario como indice de una serie regular generaria huecos. Al indexar por `t`, el `tsibble` trata la secuencia de dias *habiles* como una serie regular sin huecos, lo cual es indispensable para que funciones de *fable/feasts* (como `ETS()` o `NAIVE()`) operen correctamente.
- `as_tsibble(index = t)`: convierte el `data.frame` en un objeto `tsibble`, declarando `t` como la columna indice temporal.
- `adj_close_2015`: la serie completa, indexada desde el primer dia disponible (cercano a enero de 2015).
- `adj_close_2023`: se filtra el `tsibble` para quedarse solo con las observaciones desde el 1 de enero de 2023 en adelante, y se **reindexa** `t` desde 1 nuevamente. Esta es la unica ventana de datos utilizada en el resto del script (de ahi el comentario “unica que se usa esta semana”): se trabaja con aproximadamente 3.5 años de historia reciente, en lugar de toda la historia desde 2015, probablemente para que los modelos capturen mejor la dinamica reciente del mercado (mas relevante que el regimen de tasas de interes o volatilidad de 2015–2022).

Sustento teorico: indice ordinal vs. indice de fecha calendario. En series financieras diarias, es una practica estandar en econometria de series de tiempo indexar las observaciones por su *numero de orden de negociacion* y no por la fecha calendario, precisamente porque el proceso generador de datos (el mercado) solo se observa en dias habiles. Tratar la serie como si tuviera una periodicidad diaria calendario (incluyendo fines de semana) introduciria ceros o valores faltantes ficticios que distorsionarian la autocorrelacion y la estimacion de modelos como ETS o ARIMA.

4 Tratamiento de atipicos

El script contempla **tres versiones** de la serie `adj_close_2023` en paralelo, para poder comparar mas adelante cual tratamiento de atipicos produce mejores pronosticos:

1. **Original** (`adj_close_2023`): sin ningun tratamiento.
2. **Alterada / “trat1”** (`adj_close_2023_Alt`): atipicos corregidos automaticamente con `tsclean()`.
3. **Sin atipicos / “trat2”** (`SA_adj_close_2023`): eliminacion manual y directa de ventanas de fechas especificas.

4.1 Estrategia 1: `tsclean()` (metodo de Hyndman)

```

1 #Primera estrategia
2 #Esta funcion identifica valores outliers y propone valores
3 #optimos para la serie(Hyndman)
4 adj_close_2023_Alt = adj_close_2023
5 adj_close_2023_Alt$AdjClose = adj_close_2023_Alt$AdjClose %>%
6   tsclean()

```

Explicacion:

- Se copia la serie completa en `adj_close_2023_Alt` para no perder las demas columnas (Fecha, `t`).
- Se sobre-escribe unicamente la columna `AdjClose` aplicando `tsclean()` de la libreria `forecast`.

Sustento teorico de `tsclean()`. `tsclean()` (Hyndman) combina dos pasos:

1. **Descomposicion robusta:** la serie se descompone (usualmente via **STL**, *Seasonal-Trend decomposition using Loess*, o una suavizacion **loess** para series no estacionales) en componente de tendencia y componente residual/estacional.
2. **Deteccion de atipicos sobre el residuo:** se identifican observaciones cuyo residuo se aleja de la mediana en mas de un multiplo del **rango intercuartilico (IQR)**, un criterio robusto analogo al de un diagrama de caja (*boxplot*):

$$\text{atipico si } r_t < Q_1 - k \cdot IQR \text{ o } r_t > Q_3 + k \cdot IQR,$$

donde r_t es el residuo en el tiempo t y k es una constante fija internamente por la funcion (tipicamente relacionada con el criterio de Tukey).

3. **Reemplazo/Interpolacion:** los valores marcados como atipicos se sustituyen por un valor “razonable” obtenido mediante interpolacion lineal (o el valor ajustado de la descomposicion), de forma que la serie resultante conserva la tendencia mientras suaviza los saltos extremos, pero **no elimina observaciones** (mantiene la misma longitud n).

Esta tecnica es util cuando se sospecha de errores de medicion o eventos extremos de corta duracion (por ejemplo, choques de un dia en un *crash* de mercado) que no representan el proceso generador de datos de largo plazo, y se desea preservar el tamano muestral completo.

4.2 Estrategia 2: eliminacion manual de ventanas de fechas

```
1 #Segunda estrategia: eliminar los datos directamente en la ventana 2023
2 SA_adj_close_2023 = adj_close_2023 %>%
3   filter(Fecha < "2025-02-21" | Fecha > "2025-06-23") %>%
4   filter(Fecha < "2026-03-02" | Fecha > "2026-04-13") |>
5   mutate(t = row_number()) |>
6   as_tsibble(index = t)
```

Explicacion:

- Se filtran (**se excluyen**) dos ventanas de fechas específicas: del 21 de febrero de 2025 al 23 de junio de 2025, y del 2 de marzo de 2026 al 13 de abril de 2026.
- A diferencia de `tsclean()`, aquí las observaciones dentro de esas ventanas se **eliminan por completo** de la serie (no se interpolan ni se reemplazan), y el índice `t` se recalcula desde 1 tras el filtrado, generando así una serie *mas corta* pero “continua” en el sentido ordinal (aunque con un salto real en el calendario dentro de ella).
- Estas dos ventanas corresponden, con alta probabilidad, a periodos de **volatilidad extraordinaria** identificados visualmente por el analista (por ejemplo, episodios de incertidumbre arancelaria/comercial o correcciones fuertes de mercado en esas fechas), que el analista decidió tratar como “no representativos” del comportamiento habitual de la serie y, por tanto, remover directamente en lugar de suavizarlos.

Comparacion conceptual de las dos estrategias. La estrategia 1 (`tsclean()`) es *automatica, puntual y conservadora*: corrige observacion por observacion sin alterar el tamaño de la muestra ni requerir conocimiento experto de las fechas. La estrategia 2 es *manual, en bloque y mas agresiva*: exige que el analista identifique previamente (con apoyo grafico o conocimiento del contexto de mercado) los intervalos completos que desea excluir, y luego elimina todas las observaciones de esos intervalos. Evaluar ambas estrategias –y compararlas contra la serie original sin tratar– permite responder empíricamente la pregunta de cual tratamiento de atípicos mejora la capacidad predictiva fuera de muestra.

5 Parametros globales y funciones de evaluacion

5.1 Parametros globales

```

1 h          <- 5      # Horizonte de pronostico (dias habiles)
2 inicio_train <- 569   # Minimo de observaciones para iniciar
3 ancho_fijo  <- 45    # Tamano de ventana fija

```

- `h <- 5`: el **horizonte de pronostico**. En todo el script, cada vez que se genera un pronostico, se proyecta 5 dias habiles hacia adelante (aproximadamente una semana bursatil).
- `inicio_train <- 569`: numero minimo de observaciones (dias) requeridas antes de empezar a evaluar los modelos en el esquema de validacion *walk-forward*. Es decir, el primer conjunto de entrenamiento usa las observaciones $t = 1, \dots, 569$, y solo a partir de ahi se generan pronosticos evaluables. Este valor garantiza que el primer ajuste tenga suficiente historia (mas de dos años de datos diarios) para estimar de forma estable un modelo ETS o ARIMA.
- `ancho_fijo <- 45`: tamano de la ventana de entrenamiento cuando se usa el esquema de **ventana fija** o *rolling window* (en contraposicion al esquema **expansible**): en ese caso, cada reentrenamiento usa exactamente las ultimas 45 observaciones, en lugar de toda la historia disponible.

Sustento teorico: ventana expansible vs. ventana fija. En la validacion *walk-forward* (tambien llamada *rolling-origin evaluation*, ver Hyndman & Athanasopoulos, cap. 5), existen dos esquemas clasicos para definir el conjunto de entrenamiento en cada iteracion:

- **Expansible** (*expanding window*): el conjunto de entrenamiento siempre inicia en la primera observacion y crece a medida que avanza el origen de pronostico. Usa toda la informacion historica disponible en cada punto, a costa de que los modelos mas recientes se vean “diluidos” por datos antiguos si el proceso no es estacionario.
- **Fija** (*fixed/rolling window*): el conjunto de entrenamiento tiene un tamano constante (aqui, 45 observaciones) y se desliza hacia adelante, descartando las observaciones mas antiguas. Es mas sensible a cambios de regimen recientes (por ejemplo, cambios en la volatilidad), pero estima con menos datos y por tanto con mayor varianza.

5.2 Funcion de perdida de Winkler

```

1 #Funcion de perdida de Winkler para intervalo de confianza
2 winkler_loss = function(y_h, lim_inf, lim_sup, significancia){
3   perdida = c()
4   for (i in seq_along(y_h)) {
5     if(y_h[i]>lim_inf[i] & y_h[i]<lim_sup[i]) {
6       perdida[i] = lim_sup[i] - lim_inf[i]
7     }
8     if(y_h[i]<lim_inf[i]) {
9       perdida[i] = lim_sup[i] - lim_inf[i] + (2/significancia)*(lim_inf[i]
10         ]-y_h[i])
11     }
12     if (y_h[i]>lim_sup[i]) {

```

```

12     perdida[i] = lim_sup[i] - lim_inf[i] + (2/significancia)*(y_h[i] -
13         lim_sup[i])
14     }
15     return(perdida)
16 }

```

Explicacion: para cada observacion i , la funcion evalua si el valor real $y_h[i]$ cae dentro del intervalo $[\lim_inf[i], \lim_sup[i]]$:

- Si cae **dentro** del intervalo, la perdida es simplemente el **ancho del intervalo**.
- Si cae **por debajo** del limite inferior, se penaliza el ancho del intervalo **mas** un termino proporcional a cuanto se aleja el valor real del limite inferior, escalado por $2/\alpha$ (donde $\alpha = \text{significancia}$).
- Si cae **por encima** del limite superior, analogamente se penaliza el ancho mas un termino proporcional a la distancia por encima del limite superior.

Sustento teorico: el Winkler Score (Winkler, 1972). Esta es la implementacion textual de la **funcion de perdida de Winkler**, una regla de puntuacion propia (*proper scoring rule*) disenada especificamente para evaluar **intervalos de prediccion** (no solo pronosticos puntuales). Formalmente, para un intervalo de $(1 - \alpha) \times 100\%$ de confianza $[l_t, u_t]$ y un valor observado y_t :

$$W_t(\alpha) = \begin{cases} (u_t - l_t) & \text{si } l_t \leq y_t \leq u_t \\ (u_t - l_t) + \frac{2}{\alpha}(l_t - y_t) & \text{si } y_t < l_t \\ (u_t - l_t) + \frac{2}{\alpha}(y_t - u_t) & \text{si } y_t > u_t \end{cases}$$

Esta funcion premia intervalos **angostos** (menor ancho $u_t - l_t$ implica menor perdida cuando el valor cae dentro), pero penaliza fuertemente cuando el valor real queda **fuera** del intervalo, con una penalizacion que crece tanto con la distancia al limite incumplido como con la inverso de la significancia α (cuanto mas estricto es el nivel de confianza exigido, mayor es el castigo por fallar). Es la metrica estandar recomendada en Hyndman & Athanasopoulos para comparar pronosticos por intervalos entre distintos modelos.

5.3 Indicador de cobertura

```

1 fuera_de_int = function(y_h, lim_inf, lim_sup){
2     perdida = c()
3     for (i in seq_along(y_h)) {
4         if(y_h[i]>lim_inf[i] & y_h[i]<lim_sup[i]) {
5             perdida[i] = 0
6         }
7         else{perdida[i] = 1}
8     }
9     return(perdida)
10 }

```

Explicacion: funcion indicadora binaria: devuelve 0 si la observacion real cae dentro del intervalo, y 1 si cae fuera. Al sumar (o promediar) este indicador sobre muchas observaciones de validacion, se obtiene la **tasa empirica de no-cobertura** del intervalo, que en teoria deberia aproximarse al nivel de significancia nominal α (por ejemplo, 5% si el intervalo es del 95%). Esta es una verificacion de **calibracion** del intervalo de prediccion: intervalos bien calibrados deben fallar aproximadamente $\alpha \times 100\%$ de las veces, ni mas (intervalo demasiado angosto/optimista) ni menos (intervalo demasiado ancho y poco informativo).

5.4 Combinacion de las funciones de perdida

```

1 funciones_de_perdida = function(y_real, y_pred, significancia){
2   # Error cuadratico
3   err2 <- (y_real - y_pred)^2
4
5   # Se crea intervalo de confianza para cada paso de la validacion
6   # Asumiendo normalidad
7   limite_superior = y_pred+1.96*sqrt(err2)
8   limite_inferior = y_pred-1.96*sqrt(err2)
9
10
11   perdida_winkler = winkler_loss(y_h = y_real, lim_inf = limite_inferior,
12                                lim_sup = limite_superior, significancia =
13                                significancia)
14
15   # Queda por fuera del intervalo?
16   por_fuera = fuera_de_int(y_h = y_real, lim_inf = limite_inferior,
17                            lim_sup = limite_superior)
18
19   return(list(
20     err2 = err2,
21     perd2 = perdida_winkler,
22     por_fuera = por_fuera
23   )
24 )
25 }
```

Explicacion:

- `err2 <- (y_real - y_pred)^2`: el **error cuadratico** puntual en cada paso del horizonte, insumo basico del MSE (*Mean Squared Error*).
- `limite_superior / limite_inferior`: se construye un intervalo de confianza **ad hoc** centrado en el pronostico puntual `y_pred`, usando como medida de dispersion la raiz del error cuadratico **de esa misma observacion** (es decir, $\sqrt{(y_{\text{real}} - y_{\text{pred}})^2} = |y_{\text{real}} - y_{\text{pred}}|$), y multiplicandola por el cuantil **1.96** de la distribucion normal estandar, que corresponde a un nivel de confianza del 95% ($z_{0,975} = 1,96$).
- Es importante notar la particularidad metodologica de esta construccion: en lugar de usar una desviacion estandar estimada de forma independiente (por ejemplo, la desviacion estandar de los errores historicos de validacion), el ancho del intervalo en **cada observacion individual** se basa en el error absoluto de esa misma observacion. Esto hace que, por construccion, el intervalo **siempre contenga el valor real cuando el error es simetrico y puntual**, salvo asimetrias en la formula; en la practica esta construccion se usa aqui principalmente como

diagnostico dentro del propio ejercicio de validacion para poder calcular el Winkler score y la tasa de cobertura de forma comparable entre modelos, mas que como el metodo final de construccion de intervalos (los intervalos de prediccion **finales** del pronostico, en la Seccion 13, se construyen de otra manera: usando el MSE promedio *historico* de la validacion por horizonte, no el error puntual de cada observacion).

- `winkler_loss(...)` y `fuera_de_int(...)`: se invocan las dos funciones definidas anteriormente para calcular, respectivamente, la perdida de Winkler y el indicador de no-cobertura, usando el intervalo recién construido.
- El resultado es una lista con tres vectores paralelos: el error cuadrático, la perdida de Winkler y el indicador de fuera-de-intervalo, uno por cada observacion evaluada.

6 Funcion pronosticar_individual

Esta funcion genera un pronostico puntual a h pasos usando **un solo modelo** sobre **una sola combinacion** de anio/tratamiento, para un rango de entrenamiento arbitrario definido por `fecha_inic` y `fecha_fin`. Es la pieza basica que luego se reutiliza tanto en la generacion de pronosticos finales como (indirectamente, via una version analoga) en la evaluacion de ensambles.

6.1 Seleccion del conjunto de entrenamiento

```

1 pronosticar_individual <- function(anio = "2023",
2                                   trat = "original",
3                                   fecha_inic = "2015-01-01",
4                                   fecha_fin, modelo, h) {
5
6   if (anio == "2023") {
7     if (trat == "original") {
8       train <- adj_close_2023 %>%
9         filter(Fecha >= fecha_inic & Fecha <= fecha_fin) %>%
10        select(AdjClose)
11     }
12     if (trat == "trat1") {
13       train <- adj_close_2023_Alt %>%
14         filter(Fecha >= fecha_inic & Fecha <= fecha_fin) %>%
15        select(AdjClose)
16     }
17     if (trat == "trat2") {
18       train <- SA_adj_close_2023 %>%
19         filter(Fecha >= fecha_inic & Fecha <= fecha_fin) %>%
20        select(AdjClose)
21     }
22   }
23 }
```

Explicacion: la funcion recibe como argumentos el anio de la ventana (`anio`, aunque en la practica solo se implementa "2023"), el tratamiento de atipicos deseado (`trat`: `"original"`, `"trat1"` o `"trat2"`), las fechas de inicio y fin del entrenamiento, el nombre del modelo y el horizonte h . Segun el valor de `trat`, selecciona cual de las tres series construidas en la Seccion 3 (`adj_close_2023`, `adj_close_2023_Alt` o `SA_adj_close_2023`) usar como fuente de datos, y filtra por el rango de fechas de entrenamiento indicado, quedandose solo con la columna `AdjClose`. Este diseno modular permite reutilizar la misma funcion para probar cualquiera de los tres tratamientos simplemente cambiando el parametro `trat`.

6.2 Modelo ETS (suavizamiento exponencial)

```

1 if (modelo == "ETS") {
2   fit <- train |> model(ETS(AdjClose, opt_crit = "mse"))
3   fc <- forecast(fit, h = h)
4   y_pred <- fc$.mean
5 }
```

Explicacion: se ajusta un modelo `ETS()` de la libreria `fable` sobre la serie de entrenamiento, se pronostica h pasos adelante con `forecast()`, y se extrae la media puntual del pronostico (`fc$.mean`).

Sustento teorico: modelos ETS. ETS (*Error, Trend, Seasonal*) es un marco de modelos de **espacio de estados** para suavizamiento exponencial (Hyndman, Koehler, Ord & Snyder, 2008). Cada serie se descompone en tres componentes -error, tendencia y estacionalidad- cada uno de los cuales puede ser aditivo, multiplicativo o ausente. La función ETS() de **fable** selecciona automáticamente la especificación óptima comparando el criterio de información **AICc** entre todas las combinaciones viables. El argumento `opt_crit = "mse"` indica que, al estimar los parámetros de suavizamiento (por ejemplo α , β , γ) por máxima verosimilitud/optimización numérica, el criterio de ajuste a optimizar sea el **error cuadrático medio** en lugar de la log-verosimilitud completa u otro criterio alternativo (como "lik", ".amse" o "sigma"), lo cual alinea el criterio de estimación con la métrica de evaluación (MSE) usada posteriormente en la validación.

6.3 Modelo Naive (paseo aleatorio)

```

1  if (modelo == "Naive") {
2    fit      <- train |> model(NAIVE(AdjClose))
3    fc       <- forecast(fit, h = h)
4    y_pred   <- fc$.mean
5  }

```

Explicacion: analogo al caso ETS, pero usando el modelo NAIVE().

Sustento teorico: el modelo Naive y la hipotesis de mercados eficientes. El modelo *Naive* (paseo aleatorio sin deriva) pronostica que el valor futuro será igual al último valor observado: $\hat{y}_{t+k} = y_t$ para todo $k = 1, \dots, h$. Este es, sorprendentemente, un punto de referencia (*benchmark*) muy difícil de superar en precios de activos financieros, precisamente porque la **hipotesis de mercados eficientes** (Fama, 1970) predice que los precios siguen aproximadamente un paseo aleatorio: toda la información disponible ya está incorporada en el precio actual, por lo que no debería existir un patrón sistemático explotable para predecir el precio de mañana a partir del histórico. Incluir Naive como modelo de comparación es una práctica estándar y necesaria: si otro modelo (ETS, ARIMA, etc.) no logra superar a Naive en MSE fuera de muestra, ese modelo no aporta valor predictivo real sobre el precio.

6.4 Modelo ARMA automatico sobre log-retornos

```

1  if (grepl("AR", modelo)) {
2    y_log_ret <- diff(log(train$AdjClose))
3
4    if (modelo == "ARMA_auto") {
5      fit_ar <- auto.arima(
6        y_log_ret,
7        d      = 0,
8        seasonal = FALSE,
9        stepwise = FALSE,
10       method  = "CSS"
11      )
12    }
13
14    # Reconstruir precios desde log-retornos
15    fc_log_ret <- forecast(fit_ar, h = h)
16    log_ret_pred <- as.numeric(fc_log_ret$mean)

```

```

17     ultimo_precio <- tail(train$AdjClose, 1)
18     y_pred        <- numeric(h)
19     y_pred[1]     <- ultimo_precio * exp(log_ret_pred[1])
20     for (k in 2:h) y_pred[k] <- y_pred[k - 1] * exp(log_ret_pred[k])
21   }
22
23   return(as.numeric(y_pred))
24 }

```

Explicacion paso a paso:

- `y_log_ret <- diff(log(train$AdjClose))`: se calculan los **log-retornos** de la serie de precios: si P_t es el precio en t , entonces

$$r_t = \log(P_t) - \log(P_{t-1}) = \log\left(\frac{P_t}{P_{t-1}}\right).$$

- `grepl("AR", modelo)`: condicion que activa este bloque para cualquier nombre de modelo que contenga la subcadena `."R"` (en este script, especificamente `."RMA_auto"`).
- `auto.arima(y_log_ret, d = 0, seasonal = FALSE, stepwise = FALSE, method = "CSS")`: ajusta automaticamente un modelo **ARMA**(p, q) (sin diferenciacion adicional, pues ya se trabaja sobre los retornos, y sin componente estacional) sobre la serie de log-retornos, usando el algoritmo de Hyndman–Khandakar (2008) para seleccionar el orden (p, q) optimo segun el AICc, evaluando exhaustivamente el espacio de modelos (`stepwise = FALSE` desactiva la busqueda “paso a paso” -mas rapida pero potencialmente subs-optima- en favor de una busqueda mas completa). El argumento `method = "CSS"` indica que la estimacion de los parametros se realice por **suma condicional de cuadrados** (*Conditional Sum of Squares*), un metodo de estimacion mas rapido y robusto que la maxima verosimilitud exacta, particularmente util cuando se ajustan muchos modelos en un bucle de validacion.
- `fc_log_ret <- forecast(fit_ar, h = h)`: se pronostican los **log-retornos futuros** a h pasos.
- `ultimo_precio <- tail(train$AdjClose, 1)`: se recupera el ultimo precio observado en el conjunto de entrenamiento, necesario como punto de partida para reconstruir los precios futuros.
- El bucle

$$\hat{P}_{t+1} = P_t \cdot e^{\hat{r}_{t+1}}, \quad \hat{P}_{t+k} = \hat{P}_{t+k-1} \cdot e^{\hat{r}_{t+k}}, \quad k = 2, \dots, h$$

reconstruye los precios a partir de los retornos pronosticados, encadenando multiplicativamente cada retorno proyectado sobre el precio proyectado del paso anterior. Esto es la operacion inversa exacta de $r_t = \log(P_t/P_{t-1}) \Rightarrow P_t = P_{t-1}e^{r_t}$.

Sustento teorico: por que modelar log-retornos y no el precio directamente. Los precios de activos financieros son tipicamente **no estacionarios** (tienen raiz unitaria, tendencia estocastica), lo cual viola los supuestos de estacionariedad requeridos por los modelos ARMA puros. Los **log-retornos**, en cambio, se comportan de forma mucho mas cercana a un proceso estacionario (media y varianza aproximadamente constantes en el tiempo, aunque con posible heterocedasticidad o *volatility clustering*), lo que los hace el objeto natural de modelado en econometria financiera (siguiendo, por ejemplo, Tsay, *Analysis of Financial Time Series*). Ademas, usar logaritmos garantiza que la reconstruccion via exponencial produzca siempre **precios positivos**, lo cual no estaria garantizado si se modelara y sumara directamente el retorno simple (aritmético) sobre el precio.

Sustento teorico: modelos ARMA(p, q). Un proceso ARMA(p, q) se define como

$$r_t = c + \sum_{i=1}^p \phi_i r_{t-i} + \varepsilon_t + \sum_{j=1}^q \theta_j \varepsilon_{t-j}, \quad \varepsilon_t \sim \text{ruido blanco},$$

combinando una componente autorregresiva (AR, dependencia lineal de valores pasados de la propia serie) con una componente de medias móviles (MA, dependencia lineal de errores pasados). El algoritmo de `auto.arima()` (Hyndman & Khandakar, 2008) automatiza la selección de (p, q) -y de d , aquí fijado en 0 porque los retornos ya son estacionarios por construcción- explorando el espacio de modelos candidatos y seleccionando aquel que minimiza el AICc (criterio de información de Akaike corregido por tamaño muestral pequeño), que penaliza la sobreparametrización balanceando bondad de ajuste y parsimonia.

7 Funcion evaluar_modelos: validacion *walk-forward*

```

1  evaluar_modelos <- function(serie,
2                                tipo_ventana = "expansible",
3                                recalibrar    = TRUE,
4                                modelo       = "ETS") {
5
6    y <- serie$AdjClose
7    n <- length(y)
8    errores <- list()
9    perd_wrinkle <- list()
10   fuera_intervalo <- list()
11   i <- 1
12   t0 <- inicio_train
13
14   while ((t0 + h) <= n) {
15
16     # Definir ventana de entrenamiento
17     if (tipo_ventana == "expansible") {
18       inicio <- 1
19       fin <- t0
20     }
21     if (tipo_ventana == "fija") {
22       inicio <- max(1, t0 - ancho_fijo + 1)
23       fin <- t0
24     }
25
26     train <- serie[inicio:fin, ]

```

Explicacion:

- `y <- serie$AdjClose; n <- length(y)`: se extrae el vector de precios y su longitud total.
- Se inicializan tres listas vacias que acumularan, en cada iteracion del bucle, los errores cuadraticos, las perdidas de Winkler y los indicadores de no-cobertura.
- `t0 <- inicio_train`: el **origen de pronostico** inicial se fija en la observacion 569 (parametro global definido antes).
- El bucle `while ((t0 + h) <= n)` avanza el origen de pronostico `t0` mientras existan al menos `h` observaciones futuras disponibles para poder evaluar el pronostico contra el valor real. Esta es la esencia de la validacion *walk-forward*: en cada iteracion, se simula “estar parado” en el momento `t0`, se entrena el modelo solo con informacion hasta ese punto, se pronostica hacia adelante, y luego se compara contra lo que *realmente* ocurrio (informacion que en la practica real aun no se conoceria en el momento `t0`, pero que ya esta disponible en la muestra historica para fines de evaluacion).
- Dentro del bucle se define la **ventana de entrenamiento** segun `tipo_ventana`: si es `"expansible"`, el entrenamiento va desde la observacion 1 hasta `t0` (crece en cada iteracion); si es `"fija"`, el entrenamiento usa solo las ultimas `ancho_fijo` observaciones antes de `t0`.

```

1  # Ajustar modelo
2  if (modelo == "ETS") {
3    fit <- train |> model(ETS(AdjClose, opt_crit = "mse"))

```

```

4   }
5
6   if (modelo == "Naive") {
7     fit <- train |> model(NAIVE(AdjClose))
8   }
9
10  if (grepl("AR", modelo)) {
11    y_log_ret <- diff(log(train$AdjClose))
12
13    if (modelo == "ARMA_auto") {
14      fit_ar <- auto.arima(
15        y_log_ret,
16        d          = 0,
17        seasonal    = FALSE,
18        stepwise     = FALSE, method = "CSS"
19      )
20    }
21
22    fc_log_ret <- forecast(fit_ar, h = h)
23    log_ret_pred <- as.numeric(fc_log_ret$mean)
24    ultimo_precio <- tail(train$AdjClose, 1)
25    y_pred_ar <- numeric(h)
26    y_pred_ar[1] <- ultimo_precio * exp(log_ret_pred[1])
27    for (k in 2:h) y_pred_ar[k] <- y_pred_ar[k - 1] * exp(log_ret_pred[k
28    ])
29  }
30
31  # Pronostico
32  y_real <- y[(t0 + 1):(t0 + h)]
33
34  if (grepl("AR", modelo)) {
35    y_pred <- y_pred_ar
36  } else {
37    fc <- forecast(fit, h = h)
38    y_pred <- fc$.mean
39  }
40
41  perd = funciones_de_perdida(y_real = y_real, y_pred = y_pred,
42                             significancia = 0.05)
43  errores[[i]] <- perd$err2
44  perd_wrinkle[[i]] = perd$perd2
45  fuera_intervalo[[i]] = perd$por_fuera
46
47  t0 <- t0 + h
48  i <- i + 1
49 }

```

Explicacion:

- Segun el valor de `modelo`, se ajusta el modelo correspondiente (ETS, Naive o ARMA sobre log-retornos, replicando exactamente la misma logica de reconstruccion de precios explicada en la seccion anterior) usando **solo** los datos de la ventana de entrenamiento actual (`train`). Este re-ajuste en cada iteracion equivale a **recalibrar** el modelo en cada paso del tiempo, lo cual es coherente con el nombre del argumento `recalibrar` (aunque en el cuerpo de la funcion

no se implementa una rama alternativa que evite recalibrar; en la practica el modelo siempre se reestima en cada iteracion del bucle).

- `y_real <- y[(t0 + 1):(t0 + h)]`: se extraen los `h` valores reales inmediatamente posteriores al origen de pronostico -el “futuro verdadero” contra el cual se comparara el pronostico.
- Se genera el pronostico `y_pred` (via `forecast()` para ETS/Naive, o via la reconstruccion exponencial para ARMA).
- `funciones_de_perdida(...)`: se invoca la funcion de la Seccion 4 para calcular, en un solo paso, el error cuadratico, la perdida de Winkler y el indicador de no-cobertura para estas `h` observaciones, y los resultados se van acumulando en las listas correspondientes (indexadas por `i`, el numero de iteracion del bucle -no confundir con `t0`, el origen de pronostico).
- `t0 <- t0 + h`: el origen de pronostico avanza `h` pasos (es decir, **no hay solapamiento** entre los bloques de pronostico evaluados: se pronostica una semana, se avanza exactamente esa semana, y se repite), y `i <- i + 1` incrementa el contador de iteraciones.

```

1  errores_mat    <- do.call(rbind, errores)
2  mse_horizonte <- colMeans(errores_mat)
3  mse_global    <- mean(errores_mat)
4
5  wrinkle_mat    <- do.call(rbind, perd_wrinkle)
6  perd_wrinkle_horizonte <- colMeans(wrinkle_mat)
7  perd_wrinkle_global <- mean(wrinkle_mat)
8
9  fuera_mat     <- do.call(rbind, fuera_intervalo)
10 conteo_por_horizonte <- colSums(fuera_mat)
11 conteo_global <- sum(fuera_mat)
12
13 return(list(
14   mse_horizonte = mse_horizonte,
15   mse_global    = mse_global,
16   wrinkle_horizonte = perd_wrinkle_horizonte,
17   wrinkle_global    = perd_wrinkle_global,
18   conteo_por_horizonte = conteo_por_horizonte,
19   conteo_global = conteo_global
20 ))
21 }
```

Explicacion:

- `do.call(rbind, errores)`: convierte la lista de vectores de error (uno por cada iteracion del bucle, cada uno de longitud `h`) en una **matriz** donde cada fila es una iteracion de validacion y cada columna corresponde a un paso del horizonte ($h = 1, 2, 3, 4, 5$).
- `colMeans(errores_mat)`: promedia el error cuadratico **por columna**, obteniendo el **MSE especifico para cada paso del horizonte** (por ejemplo, el MSE promedio de pronosticar 1 dia adelante, vs. 2 dias adelante, etc.). Es una practica estandar reportar la precision del pronostico **desagregada por paso del horizonte**, ya que tipicamente el error crece con la distancia de pronostico.
- `mean(errores_mat)`: el **MSE global**, promediando sobre todas las iteraciones y todos los pasos del horizonte simultaneamente.

- Se repite exactamente la misma logica de agregacion (promedio por columna y promedio/suma global) para la perdida de Winkler (`wrinkle_mat`) y para el conteo de observaciones fuera del intervalo (`fuera_mat`), usando `colSums()/sum()` en este ultimo caso porque se trata de un **conteo** (numero total de veces que el valor real cayo fuera del intervalo), no un promedio.
- La funcion retorna una lista con las seis metricas resultantes, que se usaran posteriormente para construir tablas comparativas entre modelos y tratamientos.

Sustento teorico general de este bloque: validacion *walk-forward* u *origen movil*. A diferencia de la validacion cruzada estandar (*k-fold*) usada en datos independientes e identicamente distribuidos, en series de tiempo **no es valido** mezclar aleatoriamente observaciones pasadas y futuras entre los conjuntos de entrenamiento y validacion, pues eso violaria el orden temporal y permitiria que el modelo “vea el futuro”. El esquema implementado aqui -conocido como *time series cross-validation*, *rolling-origin evaluation* o *walk-forward validation*- respeta estrictamente el orden cronologico: el modelo solo usa datos hasta el instante t_0 para pronosticar los instantes t_0+1 hasta t_0+h , y este proceso se repite avanzando el origen en el tiempo, generando asi **multiples pronosticos fuera de muestra genuinos** sobre los cuales calcular metricas de error robustas y comparables entre modelos.

8 Funcion evaluar_ensambles

```

1 evaluar_ensambles <- function(serie, anio = "2023",
2                               tipo_ventana = "expansible",
3                               modelos,
4                               pesos) {
5
6   # -----
7   # VALIDACION
8   # -----
9
10  # los pesos suman 1
11  if (abs(sum(pesos) - 1) > 1e-6) {
12    stop(paste0(
13      "Error: los pesos deben sumar 1. Suma actual: ", round(sum(pesos),
14      6)
15    ))
16  }

```

Explicacion: antes de cualquier calculo, la funcion valida que el vector de pesos `pesos` sume 1 (con una tolerancia numerica de `1e-6` para evitar falsos positivos por errores de redondeo en punto flotante). Si la suma se desvia de 1 en mas de esa tolerancia, la funcion detiene la ejecucion con `stop()` y un mensaje informativo. Esta validacion es indispensable porque, en un **ensamble por combinacion lineal convexa** de pronosticos, los pesos deben sumar exactamente 1 para que el pronostico combinado sea una *combinacion convexa* (un promedio ponderado valido) de los pronosticos individuales, y no introduzca un sesgo sistematico de escala.

```

1  y <- serie$AdjClose
2  n <- length(y)
3  errores <- list()
4  perd_wrinkle <- list()
5  fuera_intervalo <- list()
6  i <- 1
7  t0 <- inicio_train
8
9  while ((t0 + h) <= n) {
10
11    # Definir ventana de entrenamiento
12    if (tipo_ventana == "expansible") {
13      inicio <- 1
14      fin <- t0
15    }
16    if (tipo_ventana == "fija") {
17      inicio <- max(1, t0 - ancho_fijo + 1)
18      fin <- t0
19    }
20
21    fecha_inicio = serie$Fecha[inicio]
22    fecha_fin = serie$Fecha[fin]

```

Explicacion: la estructura del bucle *walk-forward* es identica a la de `evaluar_modelos`, pero aqui, ademas de los indices `inicio/fin`, se recuperan las **fechas calendario** correspondientes al inicio y fin de la ventana de entrenamiento (`fecha_inicio`, `fecha_fin`), necesarias mas adelante para

decidir si la ventana de entrenamiento se solapa con alguno de los periodos de atipicos removidos manualmente en SA_adj_close_2023 ("trat2").

```

1  # Generar pronostico de cada modelo y ponderar
2  # y_ensemble = sum_i(pesos[i] * y_pred_i)
3  y_pred_ensemble <- numeric(h)
4
5  if (fecha_inicio >= "2025-02-21"& fecha_inicio <= "2025-06-23" |
6      fecha_inicio >= "2026-03-02"& fecha_inicio <= "2026-04-13" |
7      fecha_fin >= "2025-02-21"& fecha_fin <= "2025-06-23" |
8      fecha_fin >= "2026-03-02"& fecha_fin <= "2026-04-13") {
9      #Quita especificamente el del trat2, por la funcion de combinaciones
10     for (m in 1:nrow(modelos %>% filter(trat != "trat2"))) {
11         y_pred_m <- pronosticar_individual(anio, modelos$trat[m],
12                                             fecha_inicio,
13                                             fecha_fin,
14                                             modelos$modelo[m], h)
15         nuevos_pesos = pesos[1:nrow(modelos %>% filter(trat != "trat2"))]/
16                         sum(pesos[1:nrow(modelos %>% filter(trat != "trat2"))])
17         y_pred_ensemble <- y_pred_ensemble + nuevos_pesos[m] * y_pred_m
18     }
19 } else {
20     for (m in 1:nrow(modelos)) {
21         y_pred_m <- pronosticar_individual(anio, modelos$trat[m],
22                                             fecha_inicio,
23                                             fecha_fin,
24                                             modelos$modelo[m], h)
25         y_pred_ensemble <- y_pred_ensemble + pesos[m] * y_pred_m
26     }
27 }

```

Explicacion -esta es la parte mas sutil del script:

- El objetivo general de este bloque es calcular un **pronostico ensamblado** $\hat{y}^{\text{ens}} = \sum_m w_m \hat{y}^{(m)}$, donde cada $\hat{y}^{(m)}$ es el pronostico de uno de los modelos/tratamientos listados en el `data.frame` `modelos`, y w_m es su peso correspondiente en `pesos`.
- El problema practico es que el tratamiento "trat2" (SA_adj_close_2023) tiene **observaciones eliminadas** en dos ventanas de fechas. Si la ventana de entrenamiento actual (`fecha_inicio` a `fecha_fin`) se solapa con alguno de esos dos intervalos removidos, entonces **no es posible** construir de forma coherente el conjunto de entrenamiento para el modelo "trat2" en ese punto especifico del tiempo (la funcion `pronosticar_individual` filtraria por fechas sobre una serie que ya tiene un hueco justo ahi, produciendo un conjunto de entrenamiento mas corto o discontinuo de lo esperado).
- La condicion `if (...)` verifica si `fecha_inicio` o `fecha_fin` de la ventana de entrenamiento caen **dentro** de alguna de las dos ventanas conflictivas (2025-02-21 a 2025-06-23, o 2026-03-02 a 2026-04-13).
- Si es asi, el ensamble se calcula **excluyendo** del promedio a los modelos con `trat == "trat2"` (`modelos %>% filter(trat != "trat2")`), y los pesos de los modelos restantes se **renormalizan** dividiendolos por su propia suma (`nuevos_pesos = pesos[...] / sum(pesos[...])`), de

forma que el subconjunto de pesos usado siga sumando 1 y el pronostico siga siendo una combinacion convexa valida, ahora solo entre los modelos que *si* tienen datos de entrenamiento validos en esa ventana especifica de tiempo.

- Si la ventana de entrenamiento **no** se solapa con ninguna de las dos zonas conflictivas, se usa el ensamble completo con todos los modelos de `modelos` y sus pesos originales sin modificar.
- En ambos casos, cada pronostico individual $\hat{y}^{(m)}$ se obtiene llamando a `pronosticar_individual()` (la funcion de la Seccion 5) con el año, tratamiento, fechas y modelo especificos de la fila m de `modelos`, y se va acumulando la suma ponderada en `y_pred_ensemble`.

```

1  # Error cuadrático del ensemble
2  y_real <- y[(t0 + 1):(t0 + h)]
3
4  perd = funciones_de_perdida(y_real = y_real, y_pred = y_pred_ensemble,
5                             significancia = 0.05)
6
7  errores[[i]] <- perd$err2
8  perd_wrinkle[[i]] = perd$perd2
9  fuera_intervalo[[i]] = perd$por_fuera
10
11  t0 <- t0 + h
12  i <- i + 1
13 }
14
15 errores_mat <- do.call(rbind, errores)
16 mse_horizonte <- colMeans(errores_mat)
17 mse_global <- mean(errores_mat)
18
19 wrinkle_mat <- do.call(rbind, perd_wrinkle)
20 perd_wrinkle_horizonte <- colMeans(wrinkle_mat)
21 perd_wrinkle_global <- mean(wrinkle_mat)
22
23 fuera_mat <- do.call(rbind, fuera_intervalo)
24 conteo_por_horizonte <- colSums(fuera_mat)
25 conteo_global <- sum(fuera_mat)
26
27 return(list(
28   mse_horizonte = mse_horizonte,
29   mse_global = mse_global,
30   wrinkle_horizonte = perd_wrinkle_horizonte,
31   wrinkle_global = perd_wrinkle_global,
32   conteo_por_horizonte = conteo_por_horizonte,
33   conteo_global = conteo_global
34 ))
35 }
```

Explicacion: el resto del bucle y la agregacion final de metricas (MSE, Winkler y conteo de no-cobertura, por horizonte y globales) es **identica** a la de `evaluar_modelos`, con la unica diferencia de que aqui `y_pred` es el pronostico **combinado** del ensamble en lugar del pronostico de un modelo individual.

Sustento teorico: combinacion de pronosticos (*forecast combination*). Desde el trabajo seminal de Bates & Granger (1969), es un resultado bien establecido en la literatura de pronosticos

que combinar varios modelos -incluso modelos individualmente mediocres- frecuentemente produce un pronostico combinado con **menor error cuadratico medio** que cualquiera de los modelos individuales por separado, siempre que los errores de los modelos no esten perfectamente correlacionados (la combinacion se beneficia de la *diversificacion* del error, de forma analoga a la diversificacion de riesgo en una cartera de inversion). Aqui la combinacion se realiza sobre **el mismo tipo de modelo (ETS o ARMA_auto) pero aplicado a distintas versiones de la serie** (distintos tratamientos de atipicos), lo cual diversifica especificamente el **riesgo de especificacion respecto al tratamiento de atipicos**: si un tratamiento de atipicos concreto resulta inadecuado para un periodo particular, el ensamble amortigua ese error promediandolo con modelos entrenados sobre datos tratados de forma diferente.

9 Funcion tabla_final_MSE

```

1  #Funcion que genera tabla con MSE por horizonte y promedio
2  tabla_final_MSE <- function(resultados) {
3    map_dfr(names(resultados), function(nombre) {
4      res <- resultados[[nombre]]
5      tibble(
6        Modelo      = nombre,
7        MSE_h1       = res$mse_horizonte[1],
8        MSE_h2       = res$mse_horizonte[2],
9        MSE_h3       = res$mse_horizonte[3],
10       MSE_h4       = res$mse_horizonte[4],
11       MSE_h5       = res$mse_horizonte[5],
12       MSE_Global    = res$mse_global,
13       WRK_h1       = res$wrinkle_horizonte[1],
14       WRK_h2       = res$wrinkle_horizonte[2],
15       WRK_h3       = res$wrinkle_horizonte[3],
16       WRK_h4       = res$wrinkle_horizonte[4],
17       WRK_h5       = res$wrinkle_horizonte[5],
18       WRK_Global    = res$wrinkle_globa,
19       FUER_h1      = res$conteo_por_horizonte[1],
20       FUER_h2      = res$conteo_por_horizonte[2],
21       FUER_h3      = res$conteo_por_horizonte[3],
22       FUER_h4      = res$conteo_por_horizonte[4],
23       FUER_h5      = res$conteo_por_horizonte[5],
24       FUER_Global   = res$conteo_global
25     )
26   })
27 }

```

Explicacion: esta funcion de utilidad toma la list de resultados producida por multiples llamadas a `evaluar_modelos()` (o a `evaluar_ensambles()`) -donde cada elemento de la lista tiene un nombre identificando el escenario evaluado- y la convierte en una **tabla ordenada (tibble) con una fila por escenario**, usando `purrr::map_dfr()` para iterar sobre los nombres y apilar (*row-bind*) los resultados. Cada fila reporta, para un escenario dado:

- El MSE en cada uno de los 5 pasos del horizonte (MSE_h1...MSE_h5) y el MSE global.
- La perdida de Winkler en cada paso del horizonte y su promedio global.
- El conteo de observaciones fuera del intervalo en cada paso del horizonte y su total global.

Nota tecnica: en la linea `WRK_Global = res$wrinkle_globa`, el nombre del campo tiene una posible errata (falta la “l” final de `wrinkle_global`); dado que en R el operador `$` sobre listas hace coincidencia parcial de nombres (*partial matching*) cuando no hay ambigüedad, `res$wrinkle_globa` probablemente sigue resolviendo correctamente al campo `wrinkle_global` definido en `evaluar_modelos()`/`evaluar_ensambles()`, por lo que el codigo funciona pese a la inconsistencia tipografica. Como se indico al inicio del documento, el codigo se reproduce tal cual esta en el script original, sin corregirlo.

10 Escenarios: evaluacion de modelos individuales desde 2023

```

1  escenarios <- tribble(
2    ~nombre,          ~ventana,          ~recalibrar, ~modelo,
3    "ETS_expansible_recalibra", "expansible", TRUE,    "ETS",
4    "Naive_expansible",      "expansible", TRUE,    "Naive",
5    "ARMA_auto_expansible",  "expansible", TRUE,    "ARMA_auto"
6  )
7
8
9  resultados_2023      <- list()
10 resultados_2023_Alt <- list()
11 resultados_SA2023    <- list()
12
13 # Original
14 for (i in 1:nrow(escenarios)) {
15   esc <- escenarios[i, ]
16   cat("Ejecutando [2023 Original]:", esc$nombre, "\n")
17   resultados_2023[[esc$nombre]] <- evaluar_modelos(
18     adj_close_2023,
19     tipo_ventana = esc$ventana,
20     recalibrar   = esc$recalibrar,
21     modelo       = esc$modelo
22   )
23 }
24
25 # Alterado
26 for (i in 1:nrow(escenarios)) {
27   esc <- escenarios[i, ]
28   cat("Ejecutando [2023 Alt]:", esc$nombre, "\n")
29   resultados_2023_Alt[[esc$nombre]] <- evaluar_modelos(
30     adj_close_2023_Alt,
31     tipo_ventana = esc$ventana,
32     recalibrar   = esc$recalibrar,
33     modelo       = esc$modelo
34   )
35 }
36
37 # Sin Atipicos
38 for (i in 1:nrow(escenarios)) {
39   esc <- escenarios[i, ]
40   cat("Ejecutando [2023 SA]:", esc$nombre, "\n")
41   resultados_SA2023[[esc$nombre]] <- evaluar_modelos(
42     SA_adj_close_2023,
43     tipo_ventana = esc$ventana,
44     recalibrar   = esc$recalibrar,
45     modelo       = esc$modelo
46   )
47 }
48
49 # Tablas de MSE
50 tabla_final_2023      <- tabla_final_MSE(resultados_2023)
51 View(tabla_final_2023)
52 tabla_final_2023_Alt <- tabla_final_MSE(resultados_2023_Alt)

```

```
53 View(tabla_final_2023_Alt)
54 tabla_final_SA2023 <- tabla_final_MSE(resultados_SA2023)
55 View(tabla_final_SA2023)
```

Explicacion:

- **escenarios**: un **tibble** (creado con `tribble()`, que permite escribir tablas pequenas de forma legible por filas) que define **tres escenarios** de modelo, todos con ventana `.expansible` y `recalibrar = TRUE`: ETS, Naive y ARMA_auto.
- Se ejecutan **tres bucles for** identicos en su logica, cada uno recorriendo los 3 escenarios de **escenarios**, pero cada bucle aplicado a una version distinta de la serie: la **original** (`adj_close_2023`), la **tratada con tsclean** (`adj_close_2023_Alt`) y la **sin atipicos** (`SA_adj_close_2023`).
- En cada iteracion, `cat(...)` imprime en consola un mensaje de progreso (util quando la ejecucion es larga, pues cada llamada a `evaluar_modelos()` implica reajustar el modelo docenas de veces dentro del bucle *walk-forward*).
- Los resultados de cada combinacion (version de la serie $\times$ modelo) se almacenan en listas separadas: `resultados_2023`, `resultados_2023_Alt`, `resultados_SA2023`, cada una indexada por el nombre del escenario.
- Finalmente, `tabla_final_MSE()` convierte cada una de esas tres listas de resultados en una tabla comparativa, y `View()` (funcion de RStudio) las abre en una pestania de visualizacion tabular. Esto permite comparar, **para cada version de tratamiento de atipicos por separado**, cual de los tres modelos (ETS, Naive, ARMA_auto) tiene el menor MSE, la menor perdida de Winkler y la mejor cobertura de intervalo.

11 Validacion por combinaciones de modelos (ensambles)

```

1  #Organizar los modelos en orden de tratamiento
2  modelos <- tribble(
3    ~nombre,          ~anio,    ~trat,      ~modelo,
4    "ARMA_auto_original", "2023", "original", "ARMA_auto",
5    "ETS_trat1",        "2023", "trat1", "ETS",
6    "ARMA_auto_trat1",  "2023", "trat1", "ARMA_auto",
7    "ETS_trat2",        "2023", "trat2", "ETS",
8    "ARMA_auto_trat2",  "2023", "trat2", "ARMA_auto"
9  )
10
11 pesos <- list( media = c(1/5, 1/5, 1/5, 1/5, 1/5),
12                #inversamente proporcional al MSE global
13                inv_mse = c()
14 )

```

Explicacion:

- **modelos**: define el **conjunto de 5 modelos** que conformaran el ensamble: ARMA_auto sobre la serie original, ETS y ARMA_auto sobre la serie tratada con **tsclean** (trat1), y ETS y ARMA_auto sobre la serie sin atipicos removidos manualmente (trat2). Notese que **no se incluye Naive** ni ETS sobre la serie original, presumiblemente porque, segun los resultados de la seccion anterior, estos no fueron competitivos.
- **pesos**: una lista con dos esquemas de ponderacion a comparar:
 - **media**: pesos **iguales** (1/5 para cada uno de los 5 modelos), el esquema de combinacion mas simple posible.
 - **inv_mse**: un vector vacio que se poblara a continuacion con pesos **inversamente proporcionales al MSE global** de cada modelo (obtenido de la validacion individual de la Seccion 7).

```

1  mejores_mse <- rbind(tabla_final_2023 |>
2    filter(grepl(modelos$modelo[modelos$trat=="original"][1], Modelo)),
3    tabla_final_2023_Alt |>
4    filter(grepl(modelos$modelo[modelos$trat=="trat1"][1], Modelo)|
5      grepl(modelos$modelo[modelos$trat=="trat1"][2], Modelo)),
6    tabla_final_SA2023 |>
7    filter(grepl(modelos$modelo[modelos$trat=="trat2"][1], Modelo)|
8      grepl(modelos$modelo[modelos$trat=="trat2"][2], Modelo))
9  )
10
11 #Pesos inversamente proporcionales al MSE Global
12 for (i in 1:nrow(modelos)) {
13   pesos$inv_mse[i] = (1/mejores_mse$MSE_Global[i])/sum(1/
14     mejores_mse$MSE_Global)
15 }

```

Explicacion:

- **mejores_mse**: se construye un **data.frame** combinando (**rbind**) las filas relevantes de las tres tablas generadas en la seccion anterior (**tabla_final_2023**, **tabla_final_2023_Alt**, **tabla_final_SA2023**), filtrando en cada una solo las filas cuyo nombre de modelo coincide (via **grepl()**) con los

modelos definidos para cada tratamiento en `modelos`. El resultado es una tabla con el MSE global historico de cada uno de los 5 modelos que componen el ensamble, en el mismo orden en que aparecen en `modelos`.

- El bucle `for` calcula, para cada modelo i , el peso

$$w_i = \frac{1/\text{MSE}_i}{\sum_{j=1}^5 1/\text{MSE}_j},$$

es decir, una ponderacion **inversamente proporcional al error cuadratico medio historico** de cada modelo: los modelos con menor MSE (mejor desempenio historico) reciben **mayor peso** en el ensamble, y la division por la suma total garantiza que $\sum_i w_i = 1$, cumpliendo la restriccion validada en `evaluar_ensambles()`.

Sustento teorico: ponderacion inversa al error (*inverse-MSE weighting*). Este esquema es una aproximacion practica y simple a la combinacion **optima** de pronosticos bajo el criterio de minima varianza propuesto originalmente por Bates & Granger (1969): si los errores de los modelos fueran independientes entre si, el peso que minimiza la varianza del error combinado es inversamente proporcional a la varianza (o, de forma equivalente aqui, al MSE) de cada modelo individual. Es una alternativa razonable y computacionalmente sencilla frente a metodos mas sofisticados como la regresion de combinacion optima (que requeriria estimar tambien las covarianzas entre errores de los distintos modelos, por ejemplo mediante `nnls` para imponer no negatividad en los pesos estimados).

```

1 resultados_combinaciones = list()
2 for (i in 1:length(pesos)) {
3   pes <- pesos[[i]]
4   cat("Ejecutando:", names(pesos)[i], "\n")
5   resultados_combinaciones[[names(pesos)[i]]] <- evaluar_ensambles(
6     adj_close_2023,
7     modelos = modelos,
8     pesos = pes
9   )
10 }
11
12 tabla_final_opciones = tabla_final_MSE(resultados =
13   resultados_combinaciones)
14 View(tabla_final_opciones)

```

Explicacion: se recorre cada uno de los dos esquemas de pesos (`media` e `inv_mse`), se evalua el ensamble correspondiente con `evaluar_ensambles()` sobre `adj_close_2023` (notese que se pasa la serie **original** como `serie`, ya que dentro de `evaluar_ensambles()` cada modelo individual selecciona internamente su propia version tratada de la serie segun su columna `trat`; el argumento `serie` solo se usa para extraer los valores reales y contra los cuales comparar, que son los mismos precios sin importar el tratamiento), y se almacenan los resultados. Finalmente se construye y visualiza la tabla comparativa entre los dos esquemas de ponderacion, permitiendo decidir cual produce menor MSE/Winkler global.

12 Selecccion de modelos diferenciada por horizonte

```

1  #Probar diferentes modelos por horizonte de pronostico en la validacion
2  # =====
3
4  modelos_pasos123 = tribble(
5    ~nombre,      ~anio,    ~trat,      ~modelo,
6    "ETS_trat1",   "2023", "trat1", "ETS",
7    "ETS_trat2",   "2023", "trat2", "ETS"
8  )
9
10 modelos_pasos45 = tribble(
11   ~nombre,      ~anio,    ~trat,      ~modelo,
12   "ARMA_auto_trat1", "2023", "trat1", "ARMA_auto",
13   "ARMA_auto_trat2", "2023", "trat2", "ARMA_auto"
14 )

```

Explicacion: se plantea una estrategia mas refinada: en lugar de usar el **mismo** ensamble para los 5 pasos del horizonte, se define un conjunto de modelos distinto para los pasos cortos ($h=1,2,3$) – combinando **ETS** sobre trat1 y trat2 – y otro conjunto para los pasos largos ($h=4,5$) – combinando **ARMA_auto** sobre trat1 y trat2. Esta idea reconoce que **el mejor modelo no tiene por que ser el mismo para todos los horizontes**: por ejemplo, el suavizamiento exponencial (ETS) puede capturar mejor la inercia de muy corto plazo, mientras que un modelo ARMA sobre retornos podria ajustarse mejor a la dinamica de mas mediano plazo dentro de la semana.

```

1  mejores_mse_pasos123 <- rbind(
2    tabla_final_2023_Alt |>
3    filter(grepl(modelos_pasos123$modelo[modelos_pasos123$trat=="trat1"]
4      [1], Modelo)),
5    tabla_final_SA2023 |>
6    filter(grepl(modelos_pasos123$modelo[modelos_pasos123$trat=="trat2"]
7      [1], Modelo))
8  )
9  mejores_mse_pasos45 <- rbind(
10   tabla_final_2023_Alt |>
11   filter(grepl(modelos_pasos45$modelo[modelos_pasos45$trat=="trat1"][1],
12     Modelo)),
13   tabla_final_SA2023 |>
14   filter(grepl(modelos_pasos45$modelo[modelos_pasos45$trat=="trat2"][1],
15     Modelo))
16 )
17
18 #Pesos inversamente proporcionales al MSE Global
19 pesos_pasos123 = c()
20 for (i in 1:nrow(modelos_pasos123)) {
21   pesos_pasos123[i] = (1/mejores_mse_pasos123$MSE_Global[i])/sum(1/
22     mejores_mse_pasos123$MSE_Global)
23 }
24 pesos_pasos45 = c()
25 for (i in 1:nrow(modelos_pasos45)) {
26   pesos_pasos45[i] = (1/mejores_mse_pasos45$MSE_Global[i])/sum(1/
27     mejores_mse_pasos45$MSE_Global)
28 }

```

Explicacion: exactamente la misma logica de la seccion anterior (busqueda del MSE global historico y calculo de pesos inversamente proporcionales al MSE), mas aplicada por separado a los dos subconjuntos de modelos: `modelos_pasos123` (ETS `trat1` y `trat2`) y `modelos_pasos45` (ARMA_ `auto` `trat1` y `trat2`).

```

1 resultados_por_horizonte = list()
2 for (j in 1:2) {
3   cat("Ejecutando: PASO", j, "\n")
4   if (j == 1) {
5     resultados_por_horizonte[[j]] <- evaluar_ensambles(
6       adj_close_2023,
7       modelos = modelos_pasos123,
8       pesos = pesos_pasos123
9     )
10  } else {
11    resultados_por_horizonte[[j]] <- evaluar_ensambles(
12      adj_close_2023,
13      modelos = modelos_pasos45,
14      pesos = c(0.3, 0.7)
15    )
16  }
17 }
18
19
20 tabla_final_por_horizonte = data.frame(cbind(MSE_h1 =
21   resultados_por_horizonte[[1]]$mse_horizonte[1],
22   MSE_h2 = resultados_por_horizonte
23     [[1]]$mse_horizonte[2],
24   MSE_h3 = resultados_por_horizonte
25     [[1]]$mse_horizonte[3],
26   MSE_h4 = resultados_por_horizonte
27     [[2]]$mse_horizonte[4],
28   MSE_h5 = resultados_por_horizonte
29     [[2]]$mse_horizonte[5]))
30
31 tabla_final_por_horizonte$MSE_Global = mean(as.numeric(
32   tabla_final_por_horizonte[,1:5]))
33 View(tabla_final_por_horizonte)

```

Explicacion:

- Se evalúan **dos ensambles completos** con `evaluar_ensambles()`: el primero ($j=1$) usando `modelos_pasos123` con los pesos inversamente proporcionales al MSE calculados arriba (`pesos_pasos123`); el segundo ($j=2$) usando `modelos_pasos45`, pero aquí con pesos **fijos manualmente** en `c(0.3, 0.7)` en lugar de los pesos `pesos_pasos45` calculados previamente (una decisión explícita del analista, posiblemente porque el cálculo automático daba pesos poco razonables o porque se prefirió dar más peso, 70 %, al segundo modelo del par -ARMA_ `auto` `trat2`- por criterio experto).
- `tabla_final_por_horizonte`: se construye una tabla “híbrida” tomando el MSE de los pasos 1, 2 y 3 del **primer ensamble** (el de ETS, mejor para el corto plazo) y el MSE de los pasos 4 y 5 del **segundo ensamble** (el de ARMA_ `auto`, usado para el mediano plazo dentro de la semana). Esto materializa la idea de “diferentes modelos para diferentes horizontes”: aunque ambos ensambles se evaluaron sobre los 5 pasos completos, solo se conservan de cada uno los pasos para los que ese ensamble fue diseñado.

- **MSE_Global**: se calcula como el promedio simple de los 5 valores de MSE por horizonte ya re combinados, sirviendo como referencia global de esta estrategia hibrida final.

13 Generacion de pronosticos finales e intervalos de confianza

```

1 pronosticos_finales = numeric(5)
2 for (i in seq_along(pesos_pasos123)) {
3   y_pronos <- pronosticar_individual(anio = modelos_pasos123$anio[i],
4   trat = modelos_pasos123$trat[i],
5   fecha_inic = as.Date("2023-01-03"),
6   fecha_fin = fecha_fin, modelo = modelos_pasos123$modelo[i],
7   h = 5
8   )
9
10  pronosticos_finales = pronosticos_finales + pesos_pasos123[i]*y_pronos
11 }
12 pronosticos_finales[4:5] = c(0,0)
13 for (i in seq_along(pesos_pasos45)) {
14   y_pronos <- pronosticar_individual(anio = modelos_pasos45$anio[i],
15   trat = modelos_pasos45$trat[i],
16   fecha_inic = as.Date("2023-01-03"),
17   fecha_fin = fecha_fin, modelo = modelos_pasos45$modelo[i],
18   h = 5
19   )
20
21  pronosticos_finales[4:5] = pronosticos_finales[4:5] +
22  pesos_pasos45[i]*y_pronos[4:5]
23 }
24 pronosticos_finales

```

Explicacion paso a paso -este es el bloque que produce el pronostico definitivo a 5 dias:

- `pronosticos_finales = numeric(5)`: se inicializa un vector de 5 ceros que ira acumulando el pronostico final para cada uno de los 5 dias del horizonte.
- **Primer bucle** (sobre `modelos_pasos123`, es decir, ETS sobre `trat1` y `trat2`): para cada modelo, se llama a `pronosticar_individual()` entrenando con **todos** los datos disponibles desde el 3 de enero de 2023 (`fecha_inic`) hasta la fecha final del script (`fecha_fin`, la variable global "2026-06-19" definida en la Seccion 2 – es decir, se usa toda la historia disponible como entrenamiento, sin reservar datos para validacion, ya que la validacion fuera de muestra ya se realizo exhaustivamente en las secciones anteriores). Se pronostican los 5 dias, y se acumula la contribucion ponderada de cada modelo (`pesos_pasos123[i]`) sobre el vector `pronosticos_finales` completo (los 5 pasos).
- `pronosticos_finales[4:5] = c(0,0)`: tras el primer bucle, se **descartan explícitamente** los valores acumulados en las posiciones 4 y 5 del vector (que provienen del ensamble de ETS, pensado solo para los pasos 1 a 3), reiniciandolos a cero para que el **segundo bucle** los recalculé desde cero usando el ensamble apropiado para esos pasos.
- **Segundo bucle** (sobre `modelos_pasos45`, es decir, ARMA_auto sobre `trat1` y `trat2`): analogo al anterior, pero **solo** se acumula la contribucion ponderada sobre las posiciones 4 y 5 del vector (`y_pronos[4:5]`), usando los pesos `pesos_pasos45` definidos previamente (los inversamente proporcionales al MSE, **no** el `c(0.3, 0.7)` fijo usado solo durante la etapa de validacion de la seccion anterior).

- El resultado, `pronosticos_finales`, es el vector definitivo de 5 valores: los primeros 3 provienen del ensamble ETS(`trat1`, `trat2`) y los ultimos 2 del ensamble ARMA_au(`trat1`, `trat2`), replicando exactamente la logica hibrida por horizonte disenada y validada en la Seccion 11.

```

1 #Intervalos de confianza, usando el MSE de la validacion inversamente
2 #proporcional al MSE Global
3 mse_cols <- c("MSE_h1", "MSE_h2", "MSE_h3", "MSE_h4", "MSE_h5")
4 intervalos_normales = list()
5 for(i in seq_along(mse_cols)) {
6   intervalos_normales$limite_inferior[i] = pronosticos_finales[i] -
7     1.96*sqrt(tabla_final_por_horizonte[[mse_cols[i]]])
8   intervalos_normales$limite_superior[i] = pronosticos_finales[i] +
9     1.96*sqrt(tabla_final_por_horizonte[[mse_cols[i]]])
10 }
11
12 intervalos_normales

```

Explicacion: finalmente, se construyen los **intervalos de confianza del 95 %** para cada uno de los 5 pasos del pronostico final:

- Para cada paso $k = 1, \dots, 5$, se toma el MSE historico validado para ese paso especifico (columna `MSE_hk` de `tabla_final_por_horizonte`, la tabla hibrida construida en la Seccion 11 que combina el MSE de ETS para $k = 1, 2, 3$ y de ARMA_au para $k = 4, 5$), y se calcula su raiz cuadrada, obteniendo asi una estimacion del **error estandar de pronostico para ese horizonte especifico**, basada en el desempenio historico *fuera de muestra* de la validacion *walk-forward*.
- El intervalo se construye de la forma clasica bajo el supuesto de **normalidad** de los errores de pronostico:

$$[\hat{y}_k - 1,96 \cdot \text{RMSE}_k, \hat{y}_k + 1,96 \cdot \text{RMSE}_k], \quad \text{RMSE}_k = \sqrt{\text{MSE}_k},$$

donde $\hat{y}_k$ es el pronostico puntual final del paso k (`pronosticos_finales[i]`) y 1,96 es, nuevamente, el cuantil **97.5 %** de la normal estandar, correspondiente a un nivel de confianza del 95 % (dos colas de 2.5 % cada una).

- A diferencia del intervalo “diagnostico” construido dentro de `funciones_de_perdida()` (que usaba el error absoluto de **cada observacion individual** de validacion), este intervalo **final** usa el MSE **promedio historico agregado** por paso del horizonte, que es la practica estandar correcta para reportar un intervalo de prediccion prospectivo: se esta usando la variabilidad historica tipica del error de pronostico a k pasos como estimador de la incertidumbre esperada del pronostico futuro a ese mismo horizonte.
- El resultado, `intervalos_normales`, es una lista con dos vectores de longitud 5 (`limite_inferior` y `limite_superior`), que junto con `pronosticos_finales` conforman el resultado final y definitivo del ejercicio: un pronostico puntual a 5 dias del precio de cierre ajustado del S&P 500, acompanado de su respectivo intervalo de confianza del 95 % para cada dia, construido con base en el desempenio *fuera de muestra* validado exhaustivamente a lo largo de todo el script.

14 Conclusion metodologica

En sintesis, el script implementa un flujo de trabajo completo y riguroso para el pronostico de una serie financiera de alta frecuencia:

1. Descarga y estructuración adecuada de los datos como serie de tiempo ordinal (evitando el problema de los días no hábiles).
2. Comparación de **tres estrategias** de tratamiento de atípicos (ninguna, automática via `tsclean()`, y eliminación manual de ventanas específicas), en lugar de asumir a priori cuál es la mejor.
3. Implementación de **tres familias de modelos** (Naive como referencia base, ETS como suavizamiento exponencial, y ARMA automático sobre log-retornos como modelo econométrico clásico de series financieras).
4. Uso de **validación walk-forward** genuina, respetando el orden temporal, con dos posibles esquemas de ventana (expansible y fija).
5. Evaluación con **tres métricas complementarias**: MSE (error puntual), Winkler score (calidad del intervalo de predicción) y tasa de no-cobertura (calibración del intervalo).
6. Construcción de **ensambles** combinando modelos y tratamientos de atípicos, con dos esquemas de ponderación (igual e inversamente proporcional al MSE), incluyendo el manejo cuidadoso de las ventanas de entrenamiento que se solapan con los datos removidos manualmente.
7. Una estrategia final **diferenciada por horizonte**, usando el ensamble de ETS para los pasos cortos y el de ARMA_auto para los pasos largos, validada empíricamente antes de aplicarse al pronóstico definitivo.
8. Generación del **pronóstico final** a 5 días con sus respectivos **intervalos de confianza del 95 %**, contruidos con base en el error histórico de validación por horizonte.